

Code review — cpp-initiere

Runde de review pe proiectul tău, cea mai nouă sus.

Ultima rundă: 2026-09-09 (runda 1) · **Commit:** 2cfec23 „tema19-28” · **Compilare:**  g++ -std=c++20 -Wall -Wextra trece, 1 warning · **Configurare proiect:**  cmake refuză din prima linie · **Rulare:**  toate cele 28 de teme rulate cu inputuri alese, 5 bug-uri reproduse

Runda 1 — prima trecere prin tot `main.cpp` (2cfec23)

Ai scris 28 de teme și 10 exerciții de clasă, 455 de linii, fără să te oprești din drum: afișare → tipuri de date → operatori → `if/else` → `for` → `while`. Sintaxa e curată și consecventă — citirea de la tastatură, `%`, `/`, buclele, toate arată la fel de la prima temă până la ultima. Compilatorul, cu toate avertismentele pornite, are **o singură** obiecție în tot fișierul. La un prim modul, ăsta e un rezultat bun.

Am rulat efectiv fiecare temă, nu doar am citit-o. Ce a ieșit la iveală e **o singură slăbiciune, care se repetă de trei ori**: construiești lanțuri de condiții fără să verifici că acoperă chiar toate cazurile. De două ori în același exercițiu — pentru că `tema12` și `tema26` sunt aceeași problemă rezolvată de tine de două ori, la trei zile distanță — și amândouă variantele sunt greșite, dar **în feluri diferite**. Una tace, cealaltă minte.

Cel mai bun lucru din review-ul ăsta e că le dă râmi pe amândouă cu un singur input: `5 5 3`.

Critice

B1 — proiectul nu se configurează pe nicio altă mașină

`CMakeLists.txt:1`

```
cmake_minimum_required(VERSION 4.3)
```

Rulat pe calculatorul meu:

```
CMake Error at CMakeLists.txt:1 (cmake_minimum_required):
  CMake 4.3 or higher is required.  You are running version 4.2.3

-- Configuring incomplete, errors occurred!
```

Nici măcar nu ajunge să citească restul fișierului. La tine merge, la mine nu.

DE CE: `cmake_minimum_required` nu spune „cu ce versiune am scris eu”, ci **„refuză să continui dacă ai mai puțin de atât”**. E o barieră, nu o etichetă. CLion a scris acolo automat versiunea de CMake pe care o are el în burtă — cea mai nouă existentă — și a transformat-o, fără să te întrebe, într-

o cerință minimă. Din momentul ăla proiectul tău refuză orice mașină cu CMake mai vechi: al meu, al unui coleg, un server de teme, orice.

Regula reală e invers: pui **cea mai mică** versiune care are ce-ți trebuie ție. Tu folosești `project()`, `set(CMAKE_CXX_STANDARD 20)` și `add_executable()` — trei lucruri vechi de zece ani. **3.20** le acoperă pe toate cu mult.

FIX: `cmake_minimum_required(VERSION 3.20)`

B2 — **tema12** nu afișează NIMIC pentru **5 5 3**

`main.cpp:300–312`

Rulat cu `a=5, b=5, c=3` :

```
Introduceti un numar a
Introduceti un numar b
Introduceti un numar c
```

Atât. Programul citește cele trei numere, nu spune nimic și se termină cu succes. Fără eroare, fără crash — pur și simplu tăcere.

```
if (a>b && a>c) { ... }
else if (b>a && b>c) { ... }
else if (c>a && c>b) { ... }
else if (a==b && b==c) { cout<<"Numerele sunt egale"; }
```

DE CE: ia cazul pe rând, cu `a=5, b=5, c=3`. `a>b` e `5>5`, adică fals — deci prima ramură cade. `b>a` e tot `5>5`, fals — cade și a doua. `c>a` e `3>5`, fals — cade a treia. Ultima cere ca **toate trei** să fie egale, iar `c` e 3 — cade și ea.

Ai patru ramuri, și niciuna nu descrie situația „două numere egale, mai mari decât al treilea”. Un lanț `if / else if` nu e obligat să facă nimic: dacă nicio condiție nu e adevărată **și nu există un `else` la capăt**, execuția trece pur și simplu peste tot blocul. Compilatorul nu are cum să te avertizeze, pentru că din punctul lui de vedere „nicio ramură” e un rezultat perfect legal.

Cauza e `>` în loc de `>=`. Tu vrei să afli **cine e cel mai mare**, nu **cine e strict mai mare decât toți ceilalți** — și când doi sunt la egalitate pe primul loc, a doua întrebare nu are răspuns.

FIX: `if (a>=b && a>=c)` la toate trei ramurile. Cu `>=`, prima ramură prinde **5 5 3** și scrie corect că maximul e 5.

B3 — tema26 , același exercițiu, minte pe același input

main.cpp:417–428

tema26 e tema12 rezolvată a doua oară, pe 9 august. Rulat cu aceleași 5 5 3 :

```
Numerele sunt egale
```

Fals. 3 nu e egal cu 5. De data asta programul nu mai tace: **răspunde, și răspunde greșit**. E mai rău decât B2, pentru că un program mut te face să te uiți la el, pe când unul care răspunde sigur pe el te lasă să treci mai departe.

```
if(a>b && a>c) { ... }  
else if (b>a && b>c) { ... }  
else if (c>a && c>b) { ... }  
else { cout<<"Numerele sunt egale"<<endl; }
```

DE CE: aici ai învățat deja jumătate din lecția lui B2 — ai pus `else` la capăt, deci lanțul chiar acoperă toate cazurile și nu mai poate tăcea. Problema e **ce ai scris în el**. Un `else` înseamnă exact un singur lucru: „niciuna dintre condițiile de mai sus nu a fost adevărată”. Nu înseamnă „toate numerele sunt egale”. Tu ai presupus că singurul mod în care poți ajunge acolo e ca toate trei să fie la fel — dar mai există o cale, exact cea din B2: două egale, unul mai mic.

`else` -ul e o găleată în care cade tot ce n-ai prevăzut. Dacă îi pui o etichetă anume, ea devine automat o minciună pentru orice caz la care nu te-ai gândit.

FIX: aceeași reparație ca la B2 — `>=` în cele trei condiții. Atunci `else` chiar rămâne doar cu cazul „toate egale”, și eticheta lui devine adevărată.

+ aceeași regulă încălcată și la `main.cpp:256–270` (`tema10`) — vezi M1. Sunt trei locuri, nu două: repară toate trei, nu doar pe cele două de aici.

B4 — tema4 : media notelor pierde zecimalele

main.cpp:185

Rulat cu notele 9, 10, 10 :

```
Media notelor este :9
```

Media adevărată e 9.67. Cu notele astea, elevul din problemă pierde o zecime de punct la fiecare calcul.

```
int medianotelor=(nota1+nota2+nota3)/3;
```

DE CE: în C++, ce fel de împărțire se face nu depinde de unde pui rezultatul, ci **de ce tip sunt cele două numere pe care le împarți**. `nota1+nota2+nota3` dă `29`, un `int`. `3` e tot un `int`. Când ambii operanzi sunt întregi, `/` e împărțire întreagă: dă câtul, `9`, și aruncă restul, `2`. Zecimalele nu se pierd la atribuire — **nu s-au calculat niciodată**. Chiar dacă ai scrie `double medianotelor = (nota1+nota2+nota3)/3;`, ai obține `9.0`, pentru că `double`-ul primește un `9` deja rotunjit.

Regula asta e chiar în teoria pe care ți-ai scris-o singur, la `main.cpp:60`:

```
//    --> (int) 5/2 => 2
//    -->(double) 5/2=>2.5
```

O știi. Aici nu ai recunoscut-o pentru că întrebarea nu arăta ca `5/2` — arăta ca o medie.

FIX: ajunge să faci **un singur** operand real, iar celălalt e convertit automat:

```
double medianotelor = (nota1+nota2+nota3) / 3.0;
```

`3.0` e un `double`, deci împărțirea se face în virgulă mobilă. Un singur caracter în plus.

B5 — un `12` uitat în mijlocul răspunsului

`main.cpp:143`

Rulat cu `x=13, y=23`:

```
UC(13+23)=126
```

Ultima cifră a lui 36 e `6`, nu `126`.

```
cout<<"UC ("<<x<<"+"<<y<<"")=12"<<uc<<endl;
```

DE CE: calculul e corect — `uc` chiar conține `6`. Greșeala e în **text**: în șirul `)=12` au rămas două cifre dintr-un test vechi, lipite chiar înaintea rezultatului adevărat. `cout` nu are cum să-și dea seama: pentru el `)=12` e doar niște caractere de tipărit, exact ca `)=`. Le pune pe ecran unele după altele, iar ochiul tău citește `126` ca pe un singur număr.

E genul de bug care supraviețuiește mult, pentru că nu strică nimic în cod — strică doar adevărul afișat. Singura plasă care îl prinde e să calculezi tu răspunsul pe hârtie **înainte** de a rula, și abia apoi să compari.

FIX: `cout<<"UC("<x<<"+"<y<<"")=<uc<<endl;`

🟡 Importante

M1 — `tema10` are aceeași gaură ca `tema12`

`main.cpp:252-271`

Nota `11` → nu afișează nimic. Nota `0` → afișează corect „Respins”, dar din noroc.

Cele cinci `if`-uri acoperă 10, 9, 8, 7, 6, 5 și „sub 5”. Nu acoperă nimic peste 10, iar o notă e ceva ce vine de la tastatură, deci poate fi orice. **Este a treia apariție a regulii din B2/B3** — de aceea cere aceeași reparație: ori un `else` care spune „notă invalidă”, ori o validare `if (numar < 1 || numar > 10)` la început.

Separat: cele cinci sunt `if`-uri independente, nu `else if`. Pentru nota 10 se evaluează toate cinci, deși prima a dat deja răspunsul. Nu e greșit, e doar muncă degeaba — și te lasă expus, pentru că două condiții pot fi adevărate în același timp fără ca tu să observi.

M2 — `tema28` nu descompune durata, o convertește de trei ori

`main.cpp:446-447`

Rulat cu `3661` secunde:

Secunde:3661

Minute:61

Ore:1

Corect ar fi `1 oră, 1 minut, 1 secundă`. Tu răspunzi „durata asta are 61 de minute” — adevărat, dar nu e ce te întreabă problema. `minute = secunde/60` dă **toate** minutele, nu cele rămase după ce ai

scos orele. Ca să obții rest, îți trebuie `%`, pe care îl folosești deja corect la `tema5`, `tema6` și `tema27`:

```
int ore = o / 3600;
int minute = (o % 3600) / 60;
int secunde = o % 60;
```

M3 — `tema5`: împărțire la zero, și pe Windows chiar crapă

`main.cpp:195`

Cu `0` copii, `b/c` e împărțire la zero. Pe calculatorul meu (Mac, procesor ARM) programul a mers liniștit mai departe și a afișat `0 bomboane`. **Pe Windows, pe procesor Intel/AMD — adică la tine — același cod oprește programul cu eroare.** Nu e o diferență de C++, e o diferență de procesor: standardul spune că rezultatul e „nedefinit”, iar fiecare procesor face ce vrea.

De reținut ca regulă generală: „la mine merge” nu e o dovadă că un program e corect. Aici ajunge un `if (c == 0)` înainte de împărțire.

M4 — `tema27` crede pe cuvânt că numărul are două cifre

`main.cpp:430–439`

Cu `123` afișează „Cifra zecilor este: 12”. Cerința spune două cifre, dar cerința nu ajunge la program — utilizatorul ajunge. O validare `if (n < 10 || n > 99)` face diferența dintre un program și un program de încredere.

M5 — 455 de linii într-un singur fișier, cu `main()` recomandat manual

`main.cpp:452–455`

```
int main() {
    tema28();
    return 0;
}
```

Ca să testezi `tema12` trebuie să ștergi `tema28()`, să scrii `tema12()`, să recompilezi — și după aia să nu uiți să pui la loc. Cu 28 de teme, asta înseamnă că **majoritatea codului tău nu mai e rulat de săptămâni**. Exact așa au supraviețuit B2 și B3: ultima dată când `tema12` a rulat a fost pe 6 august.

Nu-ți trebuie foldere. Colegii tăi de C++ țin totul într-un singur proiect și **sparg codul în fișiere** `.h` , **unul per grup de teme** — `teme01-12.h` , `teme13-28.h` — iar `main.cpp` rămâne doar cu `main()` . Plus un meniu, ca să nu mai recompilezi la fiecare test:

```
int optiune;
cout<<"Ce tema rulez? "; cin>>optiune;
switch (optiune) {
    case 12: tema12(); break;
    case 26: tema26(); break;
}
```

● Cleanups

- **C1** — `main.cpp:202` : variabila `l` e declarată și nefolosită în `tema6` . E singurul warning din tot fișierul (`-Wall -Wextra`) — merită păstrat scorul ăsta.
- **C2** — temele 7–12 afișează fără `endl` la final, așa că răspunsul se lipește de linia de comandă.
- **C3** — indentarea din `tema26` (`main.cpp:417-428`) crește în trepte la fiecare `else` . Toate ramurile unui lanț sunt egale între ele, deci stau aliniate la fel — ca în `tema12` , unde ai făcut-o corect.
- **C4** — `.idea/` (setările CLion) e comis în repo. Sunt fișiere ale calculatorului tău, nu ale proiectului; locul lor e în `.gitignore` .
- **C5** — commit-urile tale sunt semnate `"Todor <\"lucator54@gmail.com\">"` , cu ghilimele curbe intrate din greșeală în nume și email. Se repară o dată, cu `git config --global user.name "Todor Luca"` și `git config --global user.email "lucator54@gmail.com"` .
- **C6** — `tema17` și `tema18` încep tabla înmulțirii de la `i=0` . Nu e greșit, dar `0 x n = 0` nu se învață la tablă; de obicei se pornește de la 1.

Before / After

#	Acum	Corect
B1	<code>cmake_minimum_required(VERSION 4.3)</code>	<code>cmake_minimum_required(VERSION 3.20)</code>
B2	<code>if (a>b && a>c)</code>	<code>if (a>=b && a>=c)</code> — la toate 3 ramurile
B3	<code>else { "Numerele sunt egale"</code> <code>} peste condiții stricte</code>	aceleași <code>>=</code> , iar <code>else</code> rămâne doar cu egalitatea reală
B4	<code>int medianotelor=</code> <code>(n1+n2+n3)/3;</code>	<code>double medianotelor=</code> <code>(n1+n2+n3)/3.0;</code>
B5	<code>cout<<"=12"<<uc;</code>	<code>cout<<"="<<uc;</code>
M2	<code>minute = secunde/60;</code>	<code>minute = (o % 3600) / 60;</code>

Q&A — răspunde-mi la astea înainte de runda 2

1. **tema12** și **tema26** sunt aceeași problemă, rezolvată de tine de două ori. Una tace pe **5 5 3**, cealaltă răspunde greșit. Care dintre ele e mai periculoasă într-un program real și de ce?
2. La B4 ți-am spus că `double medianotelor = (nota1+nota2+nota3)/3;` **tot** ar da **9**. Explică-mi în cuvintele tale de ce nu ajută că variabila e `double`.
3. Ia-ți oricare dintre cele 28 de teme și găsește-mi **un input pentru care programul tău nu afișează nimic sau afișează ceva fals**. Dacă nu găsești niciunul, spune-mi cum ai verificat.

